High-Performance Computing Assignment: Sorting Benchmarks on CPU and GPU

HPC Course — Sorting Implementations Report

Abstract

This report documents a C++20 benchmarking project that compares four sorting paths on the same input generator: serial merge sort, serial bitonic sort, OpenMP task-parallel merge sort, and CUDA bitonic sort (when the CUDA toolkit is available at configure time). The work includes a command-line benchmark driver, CMake-based builds with optional CUDA, automated correctness tests, and a shell/Python pipeline that records raw timings, aggregates statistics, and emits Markdown summaries and optional plots. We summarize the algorithms, software architecture, experimental methodology, and representative results from a full-matrix run on Linux with an NVIDIA GPU.

Contents

1 Introduction and objectives **3**

2 Theoretical background **3**
2.1 Merge sort . . . 3
2.2 Bitonic sort . . . 3
2.3 OpenMP task parallelism . . . 3
2.4 CUDA execution model . . . 3

3 Implementation overview **3**
3.1 Repository layout . . . 3
3.2 Build system (CMake) . . . 4
3.3 Serial merge sort . . . 4
3.4 OpenMP merge sort . . . 4
3.5 Serial bitonic sort . . . 5
3.6 CUDA bitonic sort . . . 5
3.7 Input generation (`utils::generate`) . . . 5
3.8 Benchmark driver (`main.cpp`) . . . 5
3.9 Correctness testing . . . 5

4 Experimental methodology **6**
4.1 Running the full matrix . . . 6
4.2 Metrics . . . 6

5 Representative results **6**
5.1 Full matrix: fastest configuration per size and distribution . . . 6
5.2 Merge track only (serial vs OpenMP) . . . 6
5.3 Bitonic track (serial merge vs serial bitonic vs CUDA) . . . 6
5.4 Speedup plots . . . 7
5.5 Timing comparison (mean wall time) . . . 9

6 Discussion **9**

7 Conclusion **10**

A Command-line usage **10**

B Building and testing **10**

1 Introduction and objectives

The goal of this assignment is to study how classical sorting algorithms behave when mapped to different parallel execution models: a single CPU thread, multiple CPU threads via OpenMP, and massively parallel SIMD-style execution on a GPU via CUDA. Rather than comparing unrelated algorithms in isolation, the project fixes a common workload generator (array size, random seed, and input distribution) so that wall-clock time differences reflect implementation and hardware mapping choices.

Concrete objectives include:

- Implement *merge sort* in serial form and again with OpenMP tasks, reusing the same merge routine for fair comparison.
- Implement *bitonic sort* on the CPU (serial) and on the GPU (CUDA), including padding to the next power of two where the algorithm requires it.
- Provide reproducible benchmarks (CLI + scripts), aggregate results into CSV, and document outcomes in generated reports.
- Validate correctness across sizes and distributions, including non-power-of-two lengths where bitonic padding matters.

2 Theoretical background

2.1 Merge sort

Merge sort divides the array into halves, sorts each half recursively, and merges the sorted halves in linear time. Its time complexity is $O(n \log n)$ in the worst case. The merge step is sequential along the merge front but offers natural parallelization at the recursion level: independent subproblems can be solved concurrently if the scheduler assigns them to different threads.

2.2 Bitonic sort

Bitonic sort is a comparison network that sorts in $O(\log^2 n)$ parallel steps for n a power of two. For general n, implementations typically pad the array to $n' = 2^{\lceil \log_2 n \rceil}$ with a sentinel larger than any real key (here, `INT32_MAX`), sort the padded region, then truncate. Bitonic sort performs more work than merge sort asymptotically on a serial machine, but its regular compare-exchange pattern maps well to GPU threads.

2.3 OpenMP task parallelism

OpenMP 3.0+ *tasks* let the programmer express recursive parallelism: a `single` thread in a parallel region creates tasks for subproblems; `taskwait` synchronizes before merging. Cutoffs are used to limit task creation overhead and to switch to a fast serial sort for tiny subarrays.

2.4 CUDA execution model

CUDA kernels launch a grid of thread blocks; threads in a block can cooperate via shared memory and barriers. The CUDA bitonic implementation uses global kernels for larger comparison stages and a shared-memory kernel for stages that fit in on-chip memory, reducing global memory traffic for those passes.

3 Implementation overview

3.1 Repository layout

- `src/` — sorting implementations and array generation utilities.

- `src/include/` — headers for serial merge, OpenMP merge, serial bitonic, CUDA bitonic, and utilities.
- `tests/sort_correctness_test.cpp` — CTest-driven correctness checks.
- `scripts/run_experiments.sh` — builds (if needed), runs the benchmark matrix, calls the analyzer.
- `scripts/analyze_performance.py` — CSV aggregation, speedup columns, plots (if Matplotlib is available), `report.md` per output directory.

3.2 Build system (CMake)

`CMakeLists.txt` requires C++20 and OpenMP. It probes for the CUDA toolkit; if found, the `.cu` bitonic source is compiled and linked against the CUDA runtime. Otherwise a stub translation unit is used so CPU-only builds still link. Two executables are produced:

- `build/Assingment` — main benchmark CLI (name as in the project).
- `build/sort_correctness_test` — correctness binary, also registered as a CTest test.

3.3 Serial merge sort

The serial merge sort uses a single auxiliary buffer `buf` of length n. The merge routine copies the left subrange into `buf`, then merges from `buf` and the right half of `arr` back into `arr`. This avoids allocating fresh left/right vectors on each merge, improving cache locality and reducing allocator pressure.

3.4 OpenMP merge sort

The parallel version reuses the same `merge` function from the serial module. Recursion uses OpenMP tasks. Two compile-time thresholds control when to parallelize and when to stop dividing the problem:

Why `TASK_CUTOFF` $\approx 2^{12}$ (4096). OpenMP tasks are not free: each task carries creation, scheduling, and bookkeeping cost. If we spawned a task for every recursive split down to very small subarrays, that overhead would dominate the actual sorting work and can even slow the program below a good serial merge sort. We therefore use the `if(right - left >= TASK_CUTOFF)` clause so that only subproblems whose span is at least about 4096 elements become tasks; smaller splits still recurse, but execute inline on the same thread without a new task. The value 2^{12} is a conventional order of magnitude—large enough that the child has substantial work to amortize task overhead, yet small enough that the recursion tree still exposes plenty of parallel subproblems for typical array sizes. It sits in the same ballpark as cutoffs used in other task-based divide-and-conquer codes (often 1k–8k elements); the exact optimum is machine- and runtime-dependent and could be tuned empirically.

Why `SORT_CUTOFF` = 32. Below a certain size, pure merge recursion is inefficient: each level pays for recursive calls and merges on tiny ranges where constant factors matter more than asymptotics. The standard fix is to *base case* into a small-array sort. Here, when `right - left < SORT_CUTOFF`, we call `std::sort` on that segment. The library implementation is highly tuned for short ranges (e.g. hybrid strategies), so this is typically faster than continuing merge sort for only a few dozen elements. The choice 32 is a common default for merge-sort leaves (values in the 16–64 range are typical); it balances avoiding deep merge chains on specks of data without making the leaf chunks so large that we waste parallel structure near the bottom of the tree.

A parallel region wraps a `single` thread that spawns the initial tasks; `taskwait` ensures both halves are sorted before merging.

3.5 Serial bitonic sort

Non-power-of-two inputs are padded with `INT32_MAX`. The classic double loop over stage length, block starts, compare distance, and indices performs compare-and-swap with direction (ascending/descending) determined by block parity. After sorting, the vector is resized back to the original length.

3.6 CUDA bitonic sort

Host code pads to the next power of two, allocates device memory, and copies the array to the GPU. For each stage $s = 1, \ldots, \log_2 n$, the implementation runs:

- **Global kernels** for passes that do not fit the shared-memory tile: each thread handles one compare-exchange pair according to stage and pass indices.
- **Shared-memory kernel** for the remaining passes within the tile size $2 \times \text{threadsPerBlock}$: data is loaded into shared memory, bitonic steps run with `__syncthreads`, then results are written back.

`threadsPerBlock` must be a positive power of two. The implementation synchronizes the device before copying results back and truncating to the original size.

3.7 Input generation (`utils::generate`)

The benchmark driver requires `-size`, `-seed`, and `-distribution`. Supported distributions include:

- `uniform` — full-range 32-bit integers.
- `gaussian` / `normal` — clipped Gaussian values.
- `nearly_sorted` — sorted base array plus $\Theta(\sqrt{n})$ random swaps.
- `reversed` — values in descending order after sorting the progression in reverse.
- `nearly_sorted_k` — windowed near-sorted variant (available in the generator; matrix scripts use the main set above).

The same seed and distribution name produce identical inputs across implementations, which is essential for fair timing comparisons.

3.8 Benchmark driver (`main.cpp`)

The program parses CLI flags, validates combinations (e.g. `-threads` only with `omp`, CUDA options only with `cuda`), generates the vector, and runs the chosen implementation for `-repeats` trials (default 1). Each trial copies the original vector (so sorts do not mutate the shared baseline across repeats), measures elapsed time with `std::chrono::high_resolution_clock`, and averages. One line is printed, including `avg_ms=...`, which the experiment scripts parse.

3.9 Correctness testing

`sort_correctness_test` runs multiple sizes (including non-powers of two), seeds, and distributions. For each case it compares outputs against a reference (serial merge) and checks non-decreasing order. CUDA tests are skipped automatically if the build uses the stub or if GPU allocation fails (e.g. no GPU in CI), so CPU paths remain validated.

4 Experimental methodology

4.1 Running the full matrix

The experiment shell script optionally ensures Matplotlib (for PNG plots), writes a system-info snapshot (uname, CPU count, GPU model if available, toolchain versions), and executes the benchmark binary for each combination of:

- array sizes (default CSV: 10^6, 2^{20}, 2^{21}, 2^{22}),
- distributions (`uniform`, `gaussian`, `nearly_sorted`, `reversed`),
- OpenMP thread counts (default: 2, 4, 8, 16),
- CUDA block sizes (default: 128, 256, 512, 1024), when CUDA is enabled,
- multiple trials per configuration (`-trials`, default 5 in the script; the sample run below used 1 trial).

Raw rows are appended to `experiments/<run_id>/raw_trials.csv`. The analyzer writes `summary.csv` with means, standard deviations, speedup versus serial merge, and optional efficiency columns; it emits `report.md` in the run directory and in category subfolders `merge_omp/` (serial merge vs OpenMP only) and `bitonic/` (serial merge vs serial bitonic vs CUDA only). PNG plots are written under `plots/` when Matplotlib imports successfully, including grouped bar charts of mean wall time per implementation under `plots/timing_comparison/` (logarithmic time axis).

4.2 Metrics

- **Mean wall time** (`avg_ms`) — primary comparison metric from the CLI average over repeats/trials.
- **Speedup vs serial merge** — ratio of serial merge mean time to the implementation mean time for the same $(n, \text{dist}, \text{seed})$.

5 Representative results

The following tables summarize run `20260328_151408` (seed 42, 1 trial per configuration) on Linux (12 logical CPUs) with an NVIDIA GeForce RTX 3050 Laptop GPU. They are illustrative; your numbers will differ with hardware, compiler flags, and `-trials`.

5.1 Full matrix: fastest configuration per size and distribution

Among serial merge, serial bitonic, OpenMP merge (all thread counts), and CUDA bitonic (all block sizes), the fastest configuration was:

Win counts in that run: OpenMP with 16 threads won 11 of 16 cells; CUDA block sizes 256 and 512 won the remainder. Serial bitonic is not competitive on the CPU for these sizes (expected given $O(n \log^2 n)$ work and bitonic structure).

5.2 Merge track only (serial vs OpenMP)

When restricted to serial merge and OpenMP merge, OpenMP with 16 threads won every cell in this run, with speedups in the 3.2×–4.3× range versus serial merge. This isolates the benefit of task-parallel merge sort without GPU participation.

5.3 Bitonic track (serial merge vs serial bitonic vs CUDA)

This subset excludes OpenMP. At $n = 10^6$, serial merge sometimes wins on `nearly_sorted` data (favorable cache and predictability for merge), and CUDA wins or ties many cases depending on

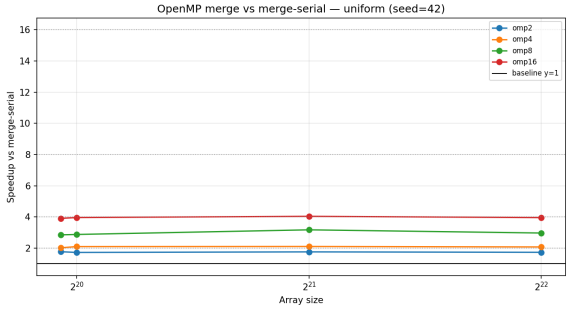
Table 1: Fastest implementation per problem size and distribution (full matrix).

n	Distribution	Fastest	Mean (ms)	vs serial merge
1 000 000	gaussian	OpenMP, 16 threads	67.14	4.06×
1 000 000	nearly_sorted	OpenMP, 16 threads	41.31	3.55×
1 000 000	reversed	OpenMP, 16 threads	38.60	3.91×
1 000 000	uniform	OpenMP, 16 threads	67.82	4.05×
1 048 576	gaussian	OpenMP, 16 threads	71.89	3.96×
1 048 576	nearly_sorted	OpenMP, 16 threads	51.75	3.24×
1 048 576	reversed	OpenMP, 16 threads	41.78	3.92×
1 048 576	uniform	OpenMP, 16 threads	74.12	3.95×
2 097 152	gaussian	CUDA, block 512	150.42	3.99×
2 097 152	nearly_sorted	OpenMP, 16 threads	87.89	3.92×
2 097 152	reversed	OpenMP, 16 threads	84.21	3.79×
2 097 152	uniform	OpenMP, 16 threads	149.75	4.04×
4 194 304	gaussian	CUDA, block 256	174.71	7.36×
4 194 304	nearly_sorted	CUDA, block 512	173.69	4.08×
4 194 304	reversed	CUDA, block 256	172.54	3.97×
4 194 304	uniform	CUDA, block 256	176.91	7.17×

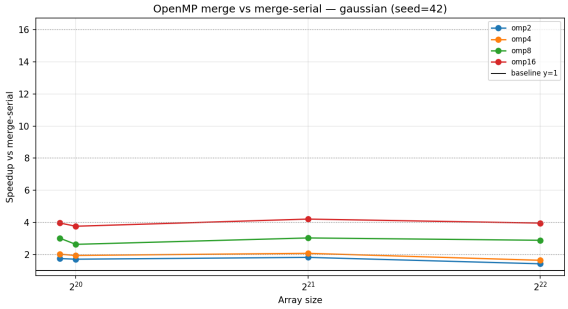
block size—but GPU launch and transfer overhead matters at smaller n. At $n = 2^{22}$, CUDA bitonic achieves large speedups over serial merge (e.g. 7.2×–7.4× for uniform/gaussian with block 256 in this run).

5.4 Speedup plots

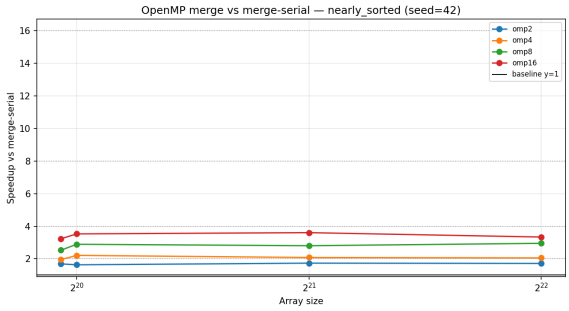
The following figures are PNG exports from `scripts/analyze_performance.py`, stored under `report/graph/` (copy from `experiments/<run_id>/plots/` after a run, or regenerate by pointing `-plot_dir` to the desired folder). They correspond to run `20260328_151408` (seed 42). The y-axis is speedup versus serial merge; dotted horizontal lines in the OpenMP plots mark ideal linear speedup at each thread count.



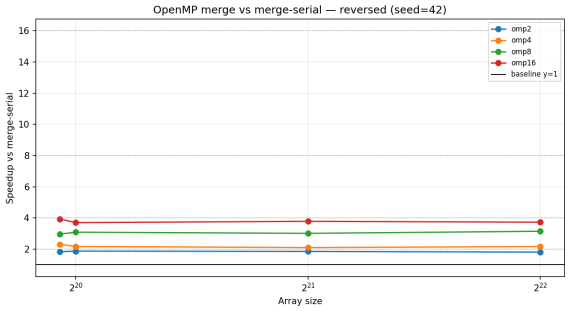
(a) `uniform`



(b) `gaussian`

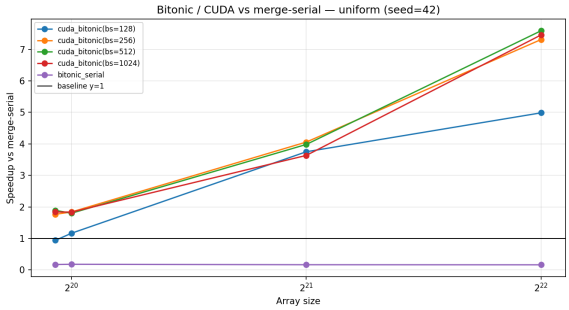


(c) `nearly_sorted`

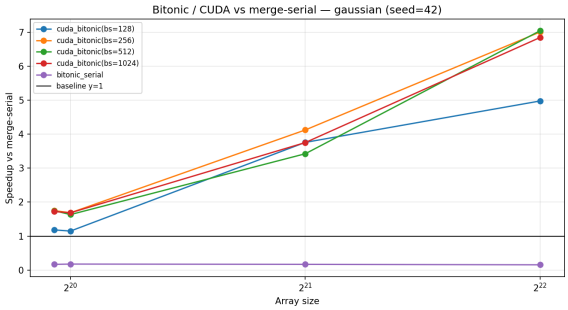


(d) `reversed`

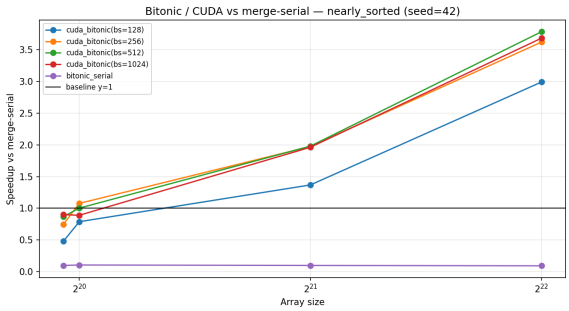
Figure 1: OpenMP merge sort: speedup vs serial merge by array size ($\log_2$ scale) and thread count.



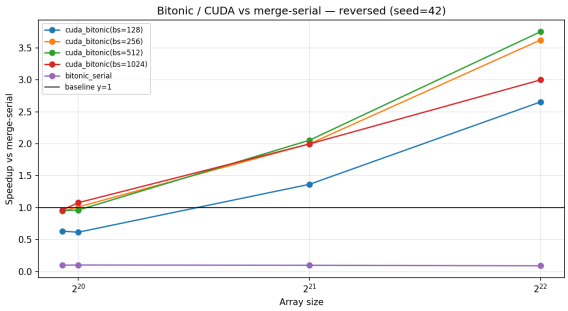
(a) `uniform`



(b) `gaussian`



(c) `nearly_sorted`

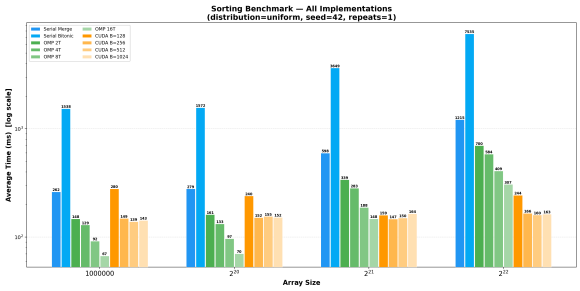


(d) `reversed`

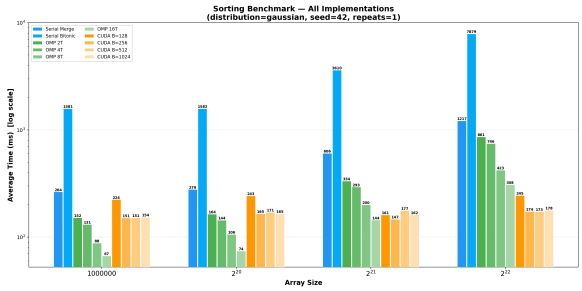
Figure 2: Serial bitonic and CUDA bitonic: speedup vs serial merge (OpenMP configurations omitted from this plot family).

5.5 Timing comparison (mean wall time)

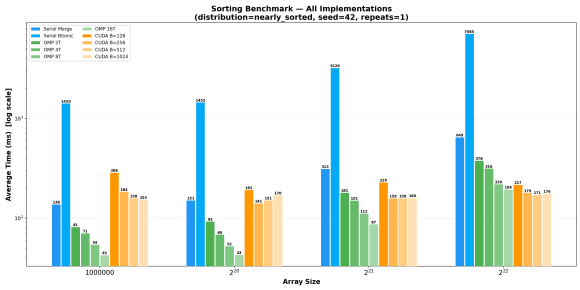
Speedup plots above are *normalized* to serial merge. The figures below show *absolute* mean wall time (milliseconds) per implementation and array size (logarithmic time axis; numeric labels on the bars).



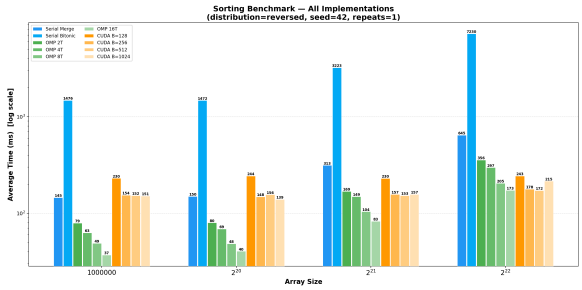
(a) `uniform`



(b) `gaussian`



(c) `nearly_sorted`



(d) `reversed`

Figure 3: Full matrix: mean wall time by array size and implementation (log scale). Missing configurations appear as pale bars.

6 Discussion

CPU vs GPU crossover. OpenMP merge sort is strong across the tested CPU sizes because merge sort is efficient sequentially and the implementation avoids excessive task creation. CUDA becomes attractive when n is large enough that kernel parallelism and throughput amortize host–device transfers and launch latency. The nearly-sorted and reversed cases often favor the CPU merge baseline at smaller n.

Bitonic on CPU. Serial bitonic is included as a reference and for correctness alignment with the GPU algorithm; it is not intended to beat merge sort on a single core for large n.

Reproducibility. Fixing `-seed` and the distribution name ensures identical inputs across `-impl` values. Increasing `-trials` and `-repeats` reduces variance from OS scheduling and turbo frequencies; reporting mean and standard deviation (in `summary.csv`) is recommended for formal submissions.

Fairness. All timed paths receive a fresh copy of the data vector per repeat inside the driver, so in-place sorts do not benefit from partially sorted state across trials.

7 Conclusion

The project delivers a cohesive toolchain: four sorting paths behind one CLI, optional CUDA integration via CMake, correctness tests with CUDA skip-on-failure behavior, and automated experiments with CSV and Markdown reporting. Empirically, OpenMP merge sort provided consistent speedups on the test machine for n up to 2^{21}, while CUDA bitonic sort dominated at $n = 2^{22}$ for several distributions once overhead was amortized. The split reports (`merge_omp`, `bitonic`) help separate "shared-memory parallel merge" effects from "GPU bitonic vs CPU baselines."

Acknowledgments

Tooling: CMake, OpenMP, optional NVIDIA CUDA toolkit, Python 3 with Matplotlib for plots.

A Command-line usage

```
./build/Assingment --size N --seed S --distribution D \
  [--impl {serial,serial_bitonic,omp,cuda}] \
  [--threads T] [--block_size B] [--repeats R]
```

Examples:

```
./build/Assingment --impl serial --size 1048576 --seed 42 \
  --distribution uniform --repeats 3
./build/Assingment --impl omp --threads 8 --size 1048576 --seed 42 \
  --distribution gaussian --repeats 3
./build/Assingment --impl cuda --block_size 512 --size 1048576 \
  --seed 42 --distribution uniform --repeats 3
```

B Building and testing

```
./build.sh
cd build && ctest --output-on-failure
./scripts/run_experiments.sh --trials 5 --seed 42
```